Department of Computer Science & Engineering (AI), Amritapuri
22AIE457 - Full Stack Development

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

For: Food Scanner & Nutrition Advisor (Web App)

Version: 1.0

Date: 05 March 2026

Objective

To clearly document all functional and non-functional requirements of an AI Food Scanner & Nutrition Advisor system built using the MERN stack, serving as a single reference for design, development, testing, and maintenance.

Team Information:

Team Member Name	Student Roll No	Project Title
A Anandakrishna	AM.SC.U4AIE23001	AI Food Scanner & Nutrition Advisor
Abhinav Nair	AM.SC.U4AIE23005	
Nakul V Sunil	AM.SC.U4AIE23048	

1 INTRODUCTION

1.1 Purpose

This document specifies the software requirements for an AI Food Scanner & Nutrition Advisor web application developed using the MERN stack.

The system will help users evaluate packaged foods using AI-based health ratings, allergen checks, and personalized nutrition advice, inspired by providing instant, unbiased ratings and detailed nutrition insights.

This SRS is intended for project guides, developers, testers, and evaluators involved in building and assessing the system.

1.2 Documentation Conventions

The following conventions are used in this document:

- FR – Functional Requirement
- NFR – Non-Functional Requirement
- API – Application Programming Interface
- JWT – JSON Web Token
- UI – User Interface
- AI – Artificial Intelligence / Machine Learning

1.3 Intended Audience

- Project Guide
- Developers
- Testers
- End Users

1.4 Product Scope

The system is a web-based AI-powered nutrition assistant that:

- Allows users to search or input packaged food products (by name, barcode, or manual entry).
- Retrieves nutrition and ingredient data from an external food database or internal dataset.
- Uses an AI scoring engine to compute health scores, allergen alerts, and color-coded recommendations (e.g., Red/Yellow/Green)
- Personalizes evaluations based on the user's health profile (e.g., diabetes, weight loss, allergies) and suggests healthier alternatives.

The application supports user registration, secure authentication, food evaluation history, progress tracking, and admin monitoring.

1.5 References

- IEEE 830 SRS Standard
- MERN Stack Documentation (MongoDB, Express.js, React.js, Node.js)
- TruthIn – Food Scanner & Nutrition Tracker (Google Play description)
- Public Food/Nutrition Databases (e.g., Open Food Facts, USDA APIs)

2 OVERALL DESCRIPTION

2.1 Product Perspective

The system follows a three-tier architecture:

- Presentation Layer: React.js SPA (Single Page Application)
- Business Logic Layer: Node.js + Express.js RESTful API
- Data Layer: MongoDB for users, profiles, products, and scan history

Additionally, the backend integrates with:

- External food/nutrition APIs (for barcode/name-based product lookup).
- An AI module (could be an internal ML model or external LLM API) that calculates health scores and generates natural language explanations and recommendations.

2.2 Product Functions

Key functions include:

- User registration, login, and profile setup (age, gender, health goals, conditions, allergies).
- Food product search by name and/or barcode number; manual entry of nutrition info if not found.
- AI-based health scoring (e.g., 0–100 score) and color-coded flags (Red/Yellow/Green).
- Allergen and ingredient alerts based on user profile (e.g., peanuts, gluten, high sugar).
- AI-generated explanations and healthier alternative suggestions.
- Scan history with analytics (graphs of average health score over time, most scanned categories).
- Admin panel for monitoring users, products, and basic analytics.

2.3 User Classes and Characteristics

- Guest User
  - Can view landing page and demo products.
  - Cannot save scans or personalized settings.
- Registered User
  - Can create a profile with health goals and allergies.
  - Can scan/search products, view AI scores, alerts, and recommendations.
  - Can view history and personal analytics.
- Admin
  - Can view all users and scan statistics.
  - Can manage flagged products and handle misuse (e.g., spam accounts).

2.4 Operating Environment

- Client Side- Modern web Browsers (Chrome, Firefox, Edge) with JavaScript enabled.
- Server Side – Node.js runtime (e.g., v18+) on Linux/Windows.
- Database – MongoDB (local or cloud, e.g., MongoDB Atlas).
- External APIs – Food/nutrition APIs and AI model endpoints accessible over HTTPS.

2.5 Design and Implementation Constraints

- Must use the MERN stack: MongoDB, Express.js, React.js, Node.js.
- Must implement JWT-based authentication for secure sessions.
- Must follow RESTful architecture for backend APIs.
- AI module must be pluggable (can switch between a simple rule-based engine and an external LLM/ML service).
- UI must be responsive across desktop, tablet, and mobile.

2.6 Assumptions and Dependencies

- Must use the MERN stack: MongoDB, Express.js, React.js, Node.js.
- Must implement JWT-based authentication for secure sessions.
- Must follow RESTful architecture for backend APIs.
- AI module must be pluggable (can switch between a simple rule-based engine and an external LLM/ML service).
- UI must be responsive across desktop, tablet, and mobile.

3 SYSTEM FEATURES (Functional Requirements)

3.1 User Registration & Profile Setup (FR1)

- The system shall allow new users to register using name, email, and password.
- The system shall validate email uniqueness.
- The system shall securely hash passwords before storing them.
- The system shall allow users to set up a health profile including:
    - Age, gender, weight, height.
    - Health goals (e.g., weight loss, muscle gain, diabetes management).
    - Dietary preferences (vegetarian, vegan, etc.).
    - Allergies (e.g., peanuts, gluten, dairy, soy).

3.2 User Login & Authentication (FR2)

- The system shall authenticate users using email and password.
- The system shall generate a JWT token upon successful login.
- The system shall use JWT for authorizing access to protected APIs.

3.3 Food Product Lookup (FR3)

- The system shall allow users to:
  - Search for products by name.
  - Enter/search by barcode number (simulating "scan").
- The system shall query an external or internal food database to retrieve nutrition facts and ingredients for the given product.
- If a product is not found, the system shall allow users to manually input nutrition details (calories, sugar, fat, protein, etc.).

3.4 AI Health Scoring & Alerts (FR4)

- The system shall pass nutrition data and user profile to the AI scoring module.
- The AI module shall compute:
  - A numeric health score (e.g., 0–100).
  - A colour category (e.g., Red = avoid, Yellow = occasional, Green = good).
- The system shall display allergen and ingredient alerts (e.g., "Contains peanuts," "High in added sugar").
- The system shall show a brief AI-generated explanation (e.g., "High sugar and saturated fat, low fibre, not recommended for diabetes").

3.5 Personalized Recommendations (FR5)

- The system shall recommend whether the product is suitable for the user's goals (e.g., weight loss, diabetes).
- The system shall suggest healthier alternatives from the database that better fit the user's profile.
- The system shall allow users to mark products as "Favourite" for quick access.

3.6 Scan History & Analytics (FR6)

- The system shall store each scan/search with date, product, health score, and user ID.
- The system shall allow users to view a history list of scanned products.
- The system shall provide analytics such as:
  - Trend of average health score over time.
  - Most frequently scanned product categories.
  - Percentage of Red/Yellow/Green items scanned.

- The system shall visualize these insights using simple charts on the dashboard (can be implemented later for bonus marks).

3.7 Admin Panel (FR7)

- The system shall store each scan/search with date, product, health score, and user ID.
- The system shall allow users to view a history list of scanned products.
- The system shall provide analytics such as:
    - Trend of average health score over time.
    - Most frequently scanned product categories.
    - Percentage of Red/Yellow/Green items scanned.
- The system shall visualize these insights using simple charts on the dashboard (can be implemented later for bonus marks).

4 EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interface

- The UI shall be implemented with React.js and be fully responsive.
- Core pages/views shall include:
    - Landing page explaining features (food scanner, allergen alerts, health scores).
    - Register and Login pages with client-side validation.
    - Dashboard with:
        - Search/barcode input.
        - Product details (nutrition table, ingredients).
        - AI score, colour indicator, alerts, explanation, and recommendations.
    - Profile page to edit health goals and allergies.
    - History & Analytics page (list + charts)
    - Admin dashboard (for admin users).

4.2 Hardware Interface

- No special hardware is required beyond a standard computer or smartphone with a browser.
- Optional: In future, camera-based barcode scanning could be integrated for mobile browsers (out of initial scope).

4.3 Software Interface

- MongoDB database for users, profiles, products, and history.
- External food/nutrition API providing product details by name/barcode.

- AI scoring service (internal module or external LLM/ML API) via HTTP.
- Browser consuming RESTful JSON APIs from Node/Express backend.

4.4 Communication Interface

- Frontend–backend communication shall use HTTP/HTTPS with JSON payloads.
- Backend–external API communication shall use HTTPS for secure transmission of product and profile data.

5 NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements (NFR1)

- For typical operations (search + scoring), the system shall respond within 3–5 seconds, depending on external API latency.
- The system shall support at least [X] concurrent users in a lab/demo environment without noticeable slowdown.

5.2 Security Requirements (NFR2)

- Passwords shall be hashed using a strong algorithm (e.g., bcrypt) before storage.
- JWT-based authentication shall be used for securing protected endpoints.
- Sensitive keys (DB URI, API keys) shall be stored in environment variables and never exposed on the client.
- Only authorized users can access their own profile and history; role-based checks for admin actions.

5.3 Reliability Requirements (NFR3)

- The system shall handle failures of external APIs gracefully, showing user-friendly error messages.
- The system shall validate API responses and handle missing or inconsistent nutrition data without crashing.

5.4 Usability Requirements (NFR4)

- The UI shall be intuitive for non-technical users, with clear labels and color-coded indicators (Red/Yellow/Green).

- The system shall provide tooltips or help text explaining health scores and color meanings.
- All forms shall provide clear validation errors (e.g., invalid email, missing fields).

5.5 Maintainability Requirements (NFR5)

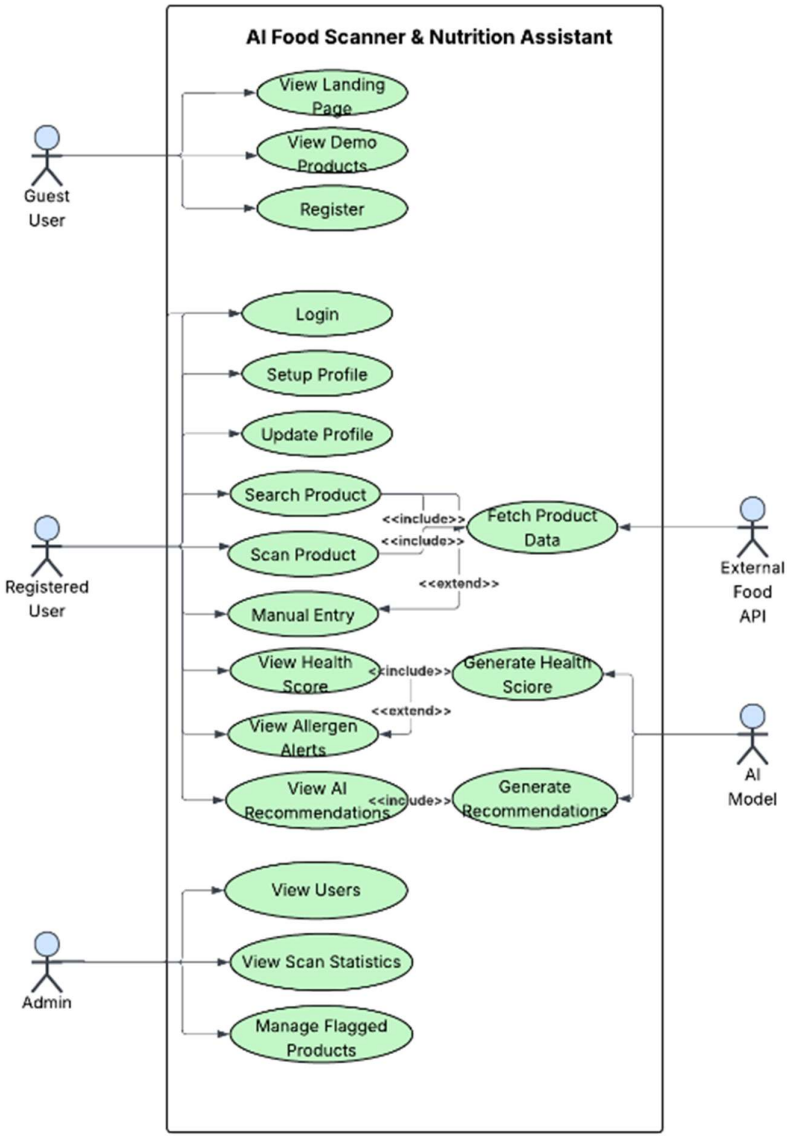
- Backend code shall follow a modular MVC-style structure (models, controllers, routes).
- Frontend shall use reusable React components and clear folder structure.
- The AI scoring logic shall be isolated in a separate service layer/module to allow future improvements.

5.6 Scalability Requirements (NFR6)

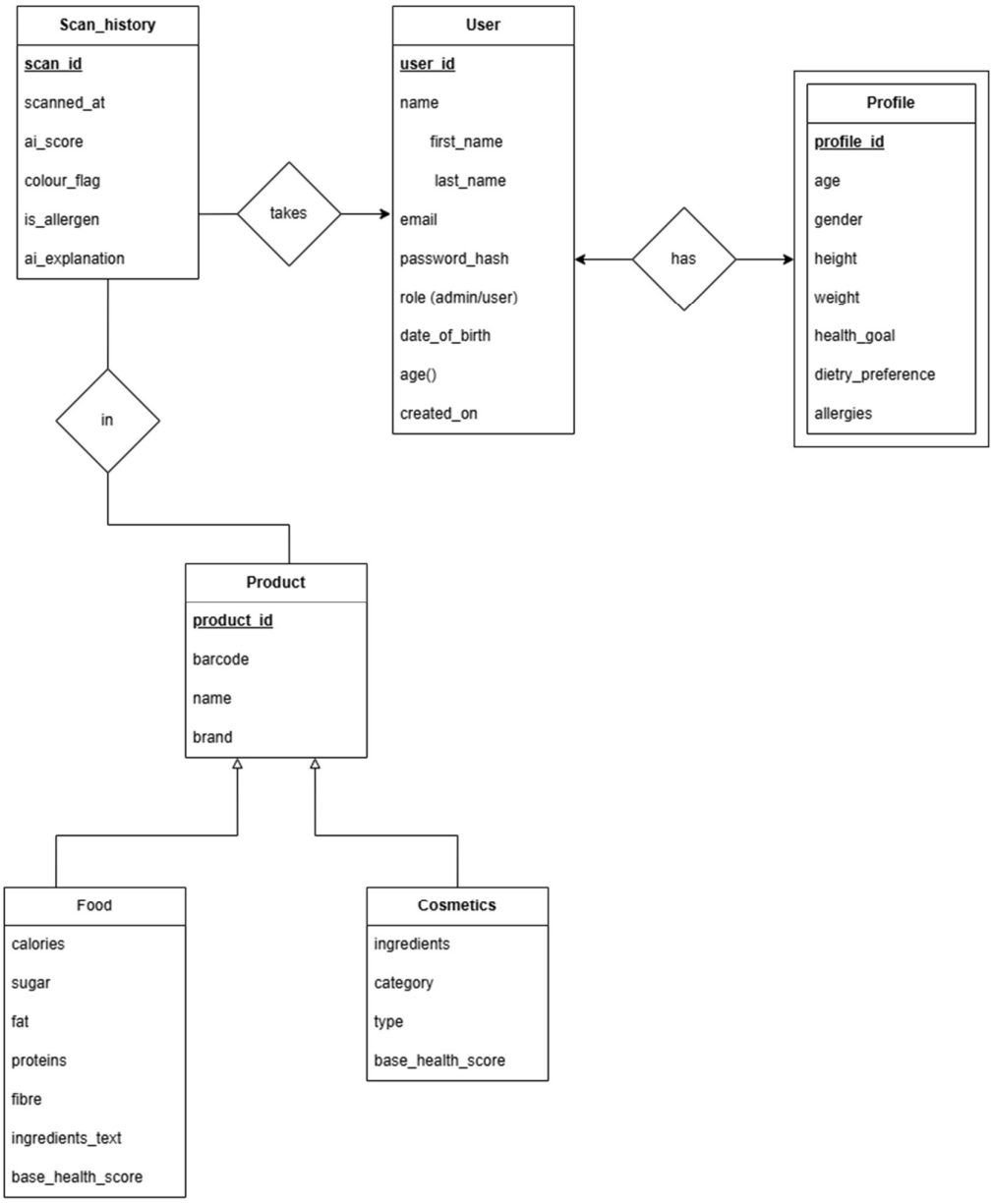
- The architecture shall support scaling horizontally by adding more backend instances and using a cloud-hosted MongoDB.
- Caching of frequent product lookups may be introduced later to reduce API calls.

6 SYSTEM MODEL

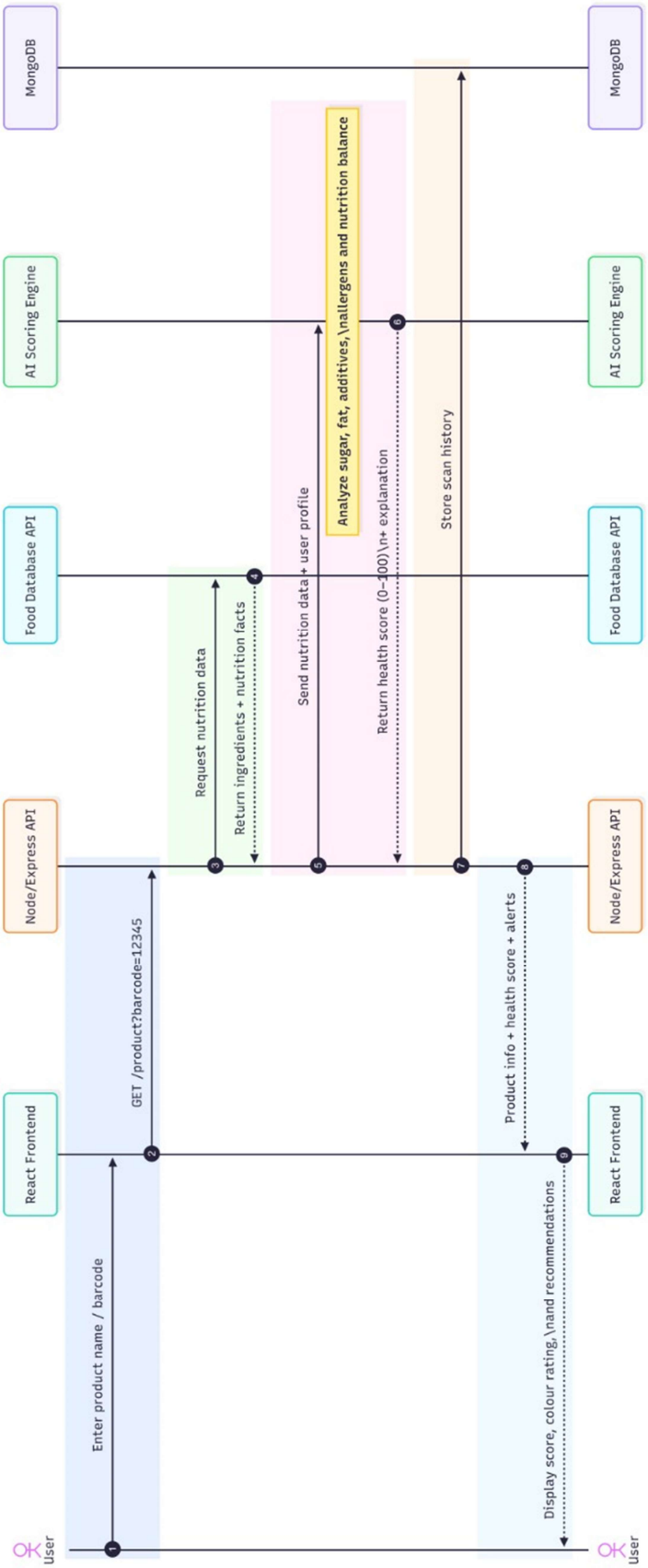
6.1 Use Case Diagram



6.2 ER Diagram



6.3 Sequence Diagram


User
React Frontend
Node/Express API
Food Database API
AI Scoring Engine
MongoDB
Enter product name / barcode
GET /product?barcode=12345
Request nutrition data
Return ingredients + nutrition facts
Send nutrition data + user profile
Analyze sugar, fat, additives, \nallergens and nutrition balance
Return health score (0–100)\n+ explanation
Store scan history
Product info + health score + alerts
Display score, colour rating, \nand recommendations
1
2
3
4
5
6
7
8
9

6.4 Architecture Diagram